

Model Deployment

1. What is Model Deployment?

Deployment is the process of taking a trained model and making it available for real users or systems to use. This means packaging the model, setting up an API or serving layer, connecting it to infrastructure, and making sure it stays reliable over time.

For data scientists and AI engineers, deployment is where the actual value gets delivered. A model sitting in a notebook helps nobody. A model running in production, serving thousands of requests, is what matters.

Deployment is not a single step — it is a pipeline

- Train and evaluate the model
- Version and register the model in a model registry (MLflow, W&B, Vertex AI Model Registry)
- Package it into a container (Docker image with all dependencies)
- Deploy to a serving infrastructure (cloud endpoint, Kubernetes cluster, serverless function)
- Expose it via an API (REST or gRPC)
- Monitor it in production and retrain when it drifts

2. Deployment Strategies

How you release a new model version matters a lot. A bad rollout can take down a product. These are the four main strategies, each with different trade-offs.

Blue-Green Deployment

You maintain two identical environments — Blue (currently live) and Green (new version). You deploy and test the new model in Green, then flip all traffic to it at once.

- Rollback is instant — just flip traffic back to Blue
- Zero downtime switchover for users
- Expensive: you need to run two full environments at the same time
- Good for: high-risk releases, compliance-sensitive systems, when you need a clean instant cutover

Watch out: For LLMs, running two full GPU environments doubles your hardware cost. Only do this if your budget allows or if the release risk is high.

Canary Deployment

You send a small percentage of real traffic to the new model version and watch how it performs before rolling out to everyone.

- Typical rollout: 5% → 10% → 25% → 50% → 100% with a wait and monitor step at each stage
- Set automatic rollback thresholds: if latency increases >20%, error rate > 1%, or quality drops — roll back automatically
- Real users hit the new model but only a small group is affected if something goes wrong
- Good for: most production releases, especially when you cannot afford full blue-green
- LLM-specific metrics to monitor during canary: hallucination rate, token cost per request, refusal rate, user satisfaction signals

Shadow Deployment (Dark Launch)

The new model receives a duplicate copy of all live traffic but its responses are NOT shown to users. You compare its outputs to the current production model silently in the background.

- Zero risk to users — they only ever see the current model's output
- Lets you validate a new model on real production traffic before anyone is affected
- Costs more compute because both models run in parallel
- Good for: validating LLM upgrades, testing new prompting strategies, comparing model versions before any real rollout

Rolling Deployment

Gradually replace old model instances with new ones, one at a time (or in small batches). At any point, some instances are running old and some new.

- No extra infrastructure needed — reuse the same cluster
- Slower rollout but resource-efficient
- Harder to rollback cleanly mid-way through
- Good for: stateless models, microservices, when cost is a constraint

Strategy	Rollback Speed	Cost	Risk to Users	Best For
Blue-Green	Instant	High (2x infra)	None	High-risk, compliance systems
Canary	Fast (config change)	Low	Very small %	Most production releases
Shadow	N/A (no user impact)	Medium	Zero	Silent validation before release
Rolling	Slow	Low	Low	Stateless models, cost-sensitive

3. Packaging and Serving

Containerization with Docker

Every production model should be packaged into a Docker container. This bundles the model weights, code, dependencies, and runtime environment together so it runs the same way everywhere — your laptop, staging, production.

- Write a Dockerfile that installs your Python environment and copies in model files
- Use multi-stage builds to keep the image small (do not include training dependencies in the serving image)
- Push to a container registry (ECR, GCR, Docker Hub) and pull from there in deployment

Serving Frameworks

Do not build a raw Flask server to serve your model. Use a proper serving framework built for this.

Framework	Best For
-----------	----------

Framework	Best For
vLLM	LLMs — continuous batching, PagedAttention, high throughput
TGI (Hugging Face)	Open-source LLMs — easy to deploy, well-documented
Triton Inference Server	Multi-model serving, GPU optimization, NVIDIA hardware
BentoML	General ML models — wraps scikit-learn, PyTorch, XGBoost etc.
FastAPI + custom	Simple models, full control over API design
TorchServe	PyTorch models, multi-model management

API Design for Model Endpoints

- Use REST for most use cases. Use gRPC when you need lower latency at high throughput.
- Always support streaming responses for LLMs — users see tokens as they arrive instead of waiting for the full response
- Version your API: `/v1/predict`, `/v2/predict`. Never break a deployed API without a new version.
- Return structured responses: prediction, confidence score, model version, request ID for tracing

4. CI/CD for ML — MLOps Pipeline

CI/CD (Continuous Integration / Continuous Delivery) means automating the process of testing and deploying code. In ML, this extends to data, models, and pipelines — not just code.

What a CI/CD pipeline for ML looks like

- Code commit triggers the pipeline (GitHub Actions, GitLab CI, Jenkins, CircleCI)
- Run unit tests on preprocessing code, feature engineering, postprocessing logic
- Run model evaluation tests — check that new model meets accuracy/latency thresholds before it can proceed
- Build a Docker image with the new model baked in
- Deploy to a staging environment and run integration tests (send real requests, check outputs)
- If all checks pass, deploy to production using canary or rolling strategy
- Log everything — what model version deployed, when, who triggered it, what tests passed

Model Registry — the Single Source of Truth

A model registry is a versioned database of all your trained models. Think of it like Git but for models.

- Every trained model gets registered with: version number, training metrics, data version it was trained on, who trained it
- Models move through stages: Staging → Production → Archived
- The CI/CD pipeline pulls the model from the registry — never deploy weights manually
- Tools: MLflow Model Registry, W&B Artifacts, Vertex AI Model Registry, SageMaker Model Registry

Key principle: Treat model versions like software releases. Every deploy is traceable. Every rollback is one registry stage change away.

5. Monitoring and Rollback

Deployment does not end when the model goes live. Models degrade over time as real-world data changes. Monitoring is what keeps you aware before users notice.

What to Monitor

What	Why	Tool Examples
Prediction distribution	Detect output drift — model starts giving different answers than before	Evidently AI, Arize, Fiddler
Input data distribution	Detect data drift — incoming data looks different from training data	Evidently AI, Great Expectations
Latency (p50, p95, p99)	Spot slowdowns before users complain	Prometheus + Grafana, Datadog
Error rate	Catch 5xx errors, timeouts, model failures	PagerDuty, Datadog, CloudWatch
Business metrics	Does the model still drive the outcome it was built for?	Custom dashboards
LLM-specific: hallucination rate, cost/request	Quality and cost control for generative models	LangSmith, Arize Phoenix, Helicone

Rollback

- Always keep the previous model version deployed and ready to receive traffic
- Rollback should be a single command or config change — not a rebuild
- Set automated rollback triggers: if error rate > threshold or latency p99 > limit, automatically revert
- After rolling back, do a post-mortem: why did the new version fail? Fix it before trying again.

Retraining and Model Refresh

- Set up automated retraining pipelines that trigger when data drift is detected or on a schedule
- Use feature stores (Feast, Tecton, Vertex AI Feature Store) to make sure training and serving use the exact same features — this is called the training-serving skew problem
- Never retrain on production data without versioning that data first

Most common deployment failure: Training-serving skew — the model was trained on cleaned, transformed data but production sends it raw data. Always keep preprocessing logic identical between training and serving.